Print Guard – Final Submission

Emily Myaskovski | Ido Katz

1. Final Product Description

- **System Architecture:** The developed system is based on a Raspberry Pi and operates on an Edge Computing architecture in order to preserve user privacy, avoid cloud dependency, and ensure low latency. The system monitors the 3D printing process locally and provides smart alerts in real time to minimize failed prints, material waste, and possible mechanical damage.

- **AI Core:** The system runs an object detection model based on the YOLOv11 architecture, optimized for edge devices. Incoming camera frames are preprocessed and resized to 640x640 pixels, balancing detection quality with Raspberry Pi memory and inference constraints. The trained model weights were exported from best.pt to best.onnx so that inference can run efficiently on edge hardware. In the final implementation, the model detects printing defects of spaghetti and stringing. For each detection, the model outputs the defect type, confidence score, and timestamp.

- **Backend Services:** The local backend service is implemented in Python using the FastAPI framework. FastAPI was selected due to its lightweight structure, high performance, and native async/await support. The backend exposes REST APIs for session management, settings updates, history retrieval, authentication, and printer control. It also includes a WebSocket layer for continuous real-time alert streaming to the dashboard.

- **Video Streaming and Inference Flow:** The backend captures frames using OpenCV, runs YOLO inference, draws detection annotations when relevant, and streams the camera feed to the browser using StreamingResponse with multipart/x-mixed-replace JPEG frames. This approach avoids heavy video encoding and is suitable for limited Raspberry Pi resources.

- **User Interface (Dashboard):** The dashboard is a responsive web-based interface implemented with HTML, CSS, and Vanilla JavaScript (ES6+). It provides live monitoring, camera feed display, current detection information, visual alerts, event timeline updates, and access to session history. The interface uses client-side JavaScript for filtering and responsive interactions.

- **Alert Mechanism and Active Control:** The system displays visual alerts on the dashboard and integrates with a Telegram bot to send immediate external notifications directly to a mobile device. In addition, the dashboard includes an active Emergency Stop feature. After user confirmation, the system sends a command through the Printer API to physically stop

the printer remotely.

- **Database and Security:** The system uses a local SQLite database to store session metadata, detection events, alerts, users, and authentication-related information. Defect snapshots are saved on the local file system, while the database stores the relevant file paths. Access to the dashboard is protected with password-based authentication, hashed passwords, token-based access control, and user-level data separation.

2. Technology Stack and Engineering Decisions

Area	Implemented Technologies	Engineering Rationale
Hardware	Raspberry Pi 4/5, camera module or USB camera, local storage	Low-cost edge device suitable for local monitoring and privacy-preserving processing.
AI / Computer Vision	YOLOv11, OpenCV, ONNX model format, 640x640 preprocessing	YOLO provides real-time object detection. ONNX improves portability and efficient inference on limited edge hardware.
Backend	Python, FastAPI, async/await, REST APIs, WebSocket, BackgroundTasks	Supports real-time communication, concurrent tasks, clean API structure, and non-blocking Telegram notifications.
Frontend	HTML, CSS, Vanilla JavaScript (ES6+), Fetch API, WebSocket API, sessionStorage	A lightweight browser-based dashboard without the overhead of a large frontend framework.
Database and Storage	SQLite, local file system, CSV export	SQLite is serverless and lightweight. Snapshots are stored as files to avoid bloating the database.
Notifications and Control	Telegram Bot API, Printer API	Telegram provides fast, free mobile alerts. Printer API enables active remote emergency stop control.

Security	Password authentication, bcrypt hashing, JWT tokens, user data separation	Protects dashboard access and restricts users to their own session history and alerts.

3. Implementation vs. Original Requirements

Most of the Must and Should requirements were implemented. In several areas, the final system was expanded beyond the original requirements. In other areas, the implementation was intentionally narrowed or modified based on engineering constraints, dataset availability, and the need to keep real-time inference stable on Raspberry Pi hardware.

3.1 Requirements Implemented

Requirement	**Implementation Status**
FR-1 Live Print Session Management	Implemented. Users can start and stop monitoring sessions, and session data is stored for history review.
FR-2 Real-Time Image/Video Capture	Implemented. Frames are captured through OpenCV and streamed to the dashboard using a lightweight JPEG stream.
FR-3 Real-Time Defect Detection	Implemented with modification. The AI pipeline performs real-time YOLO-based inference, but the final model focuses specifically on spaghetti and stringing defects.
FR-4 Dashboard Alerts and Thresholds	Implemented. Alerts are triggered when confidence exceeds the configured threshold. The dashboard includes visual alerts, and confidence threshold settings can be updated dynamically.
FR-5 Live Dashboard View	Implemented. The dashboard displays the camera feed, current detection state, latest defect information, and live alert updates.

FR-6 Run History and Reports	Implemented. The history page allows viewing and filtering previous sessions and detection events.
FR-7 Evidence Capture	Implemented. Snapshots are stored for detection events, and the database stores the snapshot paths.
NFR Security and Privacy	Implemented. Processing and storage are local by default, dashboard access is authenticated, and user data is separated.

3.2 Features Implemented Beyond the Original Requirements

- **Full User Management:** In the original requirements document, user management was defined as an optional Could requirement. In the final system, it was implemented as a complete authentication and permission system, including registration, login, password hashing, token-based access, dedicated database tables, and strict user-level data separation.

- **Emergency Stop Command:** This feature was not included in the initial requirements. It was added because the final user needs active control, not only passive monitoring. The feature allows the user to immediately stop the printer from the dashboard after confirmation, helping prevent mechanical damage and material waste.

- **Full Telegram Integration:** External notifications were originally defined as an optional stretch goal. In the final implementation, a dedicated Telegram notification service was built and connected to the alert flow, allowing real-time mobile notifications.

- **CSV Export:** The history page supports exporting filtered session and detection data to CSV, enabling offline evaluation and easier analysis of model behavior.

3.3 Requirements Modified or Not Implemented and Why

Requirement / Design Point	Final Decision and Justification
Broader Defect-Type Detection	Modified. The original requirements described several possible defect types, including bed detachment, under-extrusion, spaghetti/stringing, and unexpected stops. In the final implementation, the AI model was narrowed to spaghetti and stringing only. This decision was made due to dataset availability, annotation quality, and the need to maintain reliable real-time inference on Raspberry Pi edge

	hardware. The system architecture remains generic enough to support additional defect types in the future.
Multiple External Alert Channels	Modified. The requirements suggested several external alert alternatives such as email, Telegram, WhatsApp, or phone calls. The final implementation focuses exclusively on Telegram. Telegram was selected because it is free, fast, well documented, and does not require SMTP infrastructure or paid third-party communication providers.
Local Sound Alerts on Dashboard	Not implemented. Sound alerts were optional. The system is intended for remote browser-based monitoring, where the operator is not necessarily sitting near the Raspberry Pi. Therefore, audible alerts are delivered more effectively through the Telegram mobile application.
Automated Precision/Recall Tracking Widget	Not implemented as a live dashboard widget. The system stores detection data, confidence scores, and event history so that logs can be exported and evaluated externally. A real-time visual widget for Precision/Recall was not added in order to reduce computation and interface overhead on the edge device.
React Frontend	Changed. Although React appeared in earlier design planning, the final implemented dashboard uses HTML, CSS, and Vanilla JavaScript. This reduced complexity and kept the frontend lightweight for the project scope.

4. Final Data Flow

- User logs in through the dashboard and receives an authenticated session token.
- User starts a monitoring session from the dashboard.
- The backend captures frames from the camera using OpenCV.
- Each frame is resized and preprocessed for YOLO inference.
- The YOLOv11 ONNX model detects spaghetti or stringing defects when present.
- If the confidence score exceeds the configured threshold, the backend creates a detection event and alert record in SQLite.
- The relevant snapshot is stored on the local file system, while the database stores its path.
- The alert is streamed to the dashboard through WebSocket and sent to the user through Telegram.

- If needed, the user can trigger Emergency Stop, which sends a stop command to the printer through the Printer API.

5. Conclusion

The final system successfully implements the core project goal: a local, privacy-preserving, real-time monitoring solution for 3D printing failures using edge AI. The implemented product includes AI-based spaghetti and stringing detection, live dashboard monitoring, history review, snapshot evidence, Telegram notifications, authentication, local storage, and active Emergency Stop control. The main deviations from the original requirements were engineering decisions made to improve reliability, reduce resource consumption, and keep the system practical for Raspberry Pi deployment.